

LLM Applications in Quantum Cryptography & Quantum-Safe Software Engineering

A Categorized Literature Review with Research Synthesis and Proposal

Ourania Rahman

DREU-QIS Research Fellow

Supervisor: Dr. Lei Zhang

UMBC Emerging Software Technologies Lab

Fairleigh Dickinson University – Summer 2026

Abstract

The intersection of large language models (LLMs) and post-quantum cryptography (PQC) represents one of the most consequential emerging areas in cybersecurity research. As NIST finalizes quantum-resistant standards (FIPS 203/204/205, 2024) and federal agencies face migration mandates, the software engineering community confronts a crisis: existing codebases are pervasively quantum-vulnerable, yet automated tooling to detect, analyze, and remediate cryptographic weaknesses remains immature. Simultaneously, LLMs have demonstrated transformative capabilities in code generation, vulnerability detection, and formal reasoning, capabilities that are increasingly being applied to cryptographic contexts.

This literature review categorizes and synthesizes the current body of research at the LLM $\times$ cryptography $\times$ quantum-safe software intersection. We identify six distinct research clusters: (1) LLM-assisted static analysis and crypto detection, (2) fine-tuned LLMs for secure code generation, (3) LLM benchmarking in cryptographic domains, (4) multi-agent LLM systems for security, (5) LLM-driven PQC hardware acceleration, and (6) quantum-safe software engineering frameworks (including Dr. Zhang's EST Lab work). We assess each cluster's methodology, findings, and limitations, and conclude with a synthesis of what remains unsolved and a concrete research proposal for summer 2026.

1. Introduction & Motivation

Two independent technological trajectories are converging to create an urgent research problem. First, the quantum computing threat: Shor's algorithm, executed on a cryptographically relevant quantum computer (CRQC), breaks RSA, ECDSA, and Diffie-Hellman, the foundations of modern internet security. The "harvest now, decrypt later" (HNDL) strategy means adversaries are already collecting encrypted traffic today. NIST's August 2024 finalization of FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) provides quantum-resistant replacements, but migration of the world's software to these standards is a massive, largely unsolved engineering challenge.

Second, the LLM revolution: large language models have demonstrated unprecedented capabilities in code understanding, generation, and vulnerability analysis. Tools like GitHub Copilot, CodeBERT, and GPT-4 are transforming software engineering practice, but their application to quantum-safe migration specifically is nascent, fragmented, and largely unevaluated.

The central question of this literature review, and the proposed summer research, is: How can LLMs accelerate, improve, or enable the migration of classical software to post-quantum cryptography? And where do current approaches fall short?

Research Context: This review is produced in support of the DREU-QIS summer fellowship under Dr. Lei Zhang at UMBC's EST Lab, whose work on quantum-safe software engineering (IBM Db2 case study, PQC library survey, AQuA framework) forms the applied anchor of this review.

2. Review Methodology

This review covers papers published from 2022 through mid-2026, with emphasis on 2024–2026 to capture the most current work. Sources include IEEE S&P, USENIX Security, CCS, ICSE, QCE, SANER, EMNLP, arXiv preprints, and NIST publications. Papers were selected for direct relevance to at least two of the three themes: large language models, cryptography/PQC, and software engineering/security. The review is organized by LLM methodology rather than application domain, to surface transferable technical patterns across papers. The cluster totals below reflect the full paper collection; a representative subset of each cluster is reviewed in detail in Section 3.

Cluster	# Papers	Primary Venues
LLM-Assisted Static Analysis & Crypto Detection	7	ICSE, arXiv, ACSAC
Fine-Tuned LLMs for Secure Code Generation	8	CCS, IEEE TSE, USENIX
LLM Benchmarking in Cryptographic Domains	4	EMNLP, arXiv
Multi-Agent LLM Systems for Security	5	arXiv, ACM TOSEM
LLM-Driven PQC Hardware Acceleration	3	arXiv, IEEE
Quantum-Safe SE Frameworks (incl. Zhang et al.)	6	IST, SANER, ICSE, QCE

3. Literature Review by Cluster

Cluster A: LLM-Assisted Static Analysis & Crypto Detection

This is the cluster most directly relevant to quantum-safe migration. These papers apply LLMs to the problem of finding, classifying, and contextualizing cryptographic usage in codebases, which is the first step in any PQC migration pipeline.

A1. Quantum-Safe Code Auditor (Shaw, 2026)

Shaw, A. (2026). Quantum-Safe Code Auditing: LLM-Assisted Static Analysis and Quantum-Aware Risk Scoring for Post-Quantum Cryptography Migration. arXiv:2604.00560.

Method: Hybrid system combining regex-based detection of 15 classes of quantum-vulnerable primitives with LLM-assisted contextual enrichment and a Variational Quantum Eigensolver (VQE) model for risk scoring. Evaluated on 5,775 findings across five open-source libraries (python-rsa, python-ecdsa, python-jose, node-jsonwebtoken, Bouncy Castle Java). Achieved 71.98% precision, 100% recall, F1 = 83.71% on a stratified sample of 602 labelled instances.

Significance: The first paper to combine LLM analysis with actual quantum computing (VQE) in a PQC migration tool. The 100% recall is critical for security contexts; missing a vulnerable primitive is worse than a false positive. However, 72% precision means nearly a third of flagged items require manual review, which may be impractical at enterprise scale.

Limitations: Evaluated only on small open-source libraries; no evaluation on large enterprise codebases (e.g., the scale of IBM Db2). The VQE component adds complexity but its advantage over classical risk scoring is not clearly demonstrated. LLM component is not fine-tuned on cryptographic code.

A2. Detecting Cryptographically Relevant Packages with Collaborative LLMs (2026)

Hirsch, E., Raab, K., Bauer, T. J., & Loebenberg, D. (2026). Detecting Cryptographically Relevant Software Packages with Collaborative LLMs. ICISP 2026. arXiv:2603.07204.

Method: Local, on-premises LLM framework (privacy-preserving, offline) for identifying software packages that use cryptography, as a prerequisite for generating Cryptographic Bill of Materials (CBOM). Uses prompt engineering and iterative optimization. Multi-model ensemble approach that combines outputs from multiple local LLMs (GPT4All, Ollama) through majority voting, evaluated on over 65,000 Fedora Linux packages.

Significance: Addresses a critical practical constraint: enterprises cannot send proprietary codebases to cloud LLM APIs. The on-premises framing makes this directly deployable in government and defense contexts (NSA CNSA 2.0 compliance). CBOM generation is explicitly required by CISA/NSA/NIST quantum-readiness guidance.

Limitations: Package-level detection is coarser than function-level. Prompt engineering is described as "iterative" but no formal methodology is given. No comparison to traditional SAST tools.

A3. IRIS: LLM-Assisted Static Analysis (Li, Dutta & Naik, 2025)

Li, Z., Dutta, S., & Naik, M. (2025). IRIS: LLM-assisted static analysis for detecting security vulnerabilities. ICLR 2025.

Method: Combines traditional static analysis with LLM reasoning to reduce false positives. Static analysis generates candidate vulnerability sites; LLM provides contextual judgment on whether each candidate is a genuine vulnerability. Addresses the fundamental precision problem of rule-based tools.

Significance: The hybrid static+LLM architecture is the most pragmatic model for enterprise deployment: pure LLM analysis is too slow and costly at scale; pure static analysis has too many false positives. IRIS points toward the right integration pattern for PQC migration tools.

Limitations: Evaluated on general vulnerability detection, not cryptographic-specific patterns. Unknown performance on PQC-specific detection tasks (quantum-vulnerable API detection has different characteristics than general buffer overflows).

A4. ChatGPT for Cryptographic Misuse Detection (Firouzi & Ghafari, 2024)

Firouzi, E., & Ghafari, M. (2024). ChatGPT's Potential in Cryptography Misuse Detection: A Comparative Analysis with Static Analysis Tools. ESEM 2024.

Method: Comparative study of GPT-4 (zero-shot) vs. traditional static analysis tools (CryptoGuard, FindSecBugs) for detecting cryptographic API misuse in Java. Tests on real-world Java projects.

Significance: One of the first direct empirical comparisons of LLMs vs. SAST for crypto-specific detection. Finding: GPT-4 catches semantically complex misuses that static tools miss (e.g., incorrect key derivation logic) but misses syntactic patterns that static tools reliably catch. This complementarity directly motivates hybrid approaches.

Limitations: Zero-shot only, with no fine-tuning on cryptographic misuse patterns. GPT-4 API, not local deployment. Only tested on Java. Does not cover PQC-specific patterns (only classical crypto misuse).

A5. FoC: Cryptographic Function Identification in Binaries (Shang et al., 2024)

Shang, X., Chen, G., Cheng, S., Zhang, Y., Zhang, W., & Yu, N. (2024). FoC: Figure out the Cryptographic Functions in Stripped Binaries with LLMs. arXiv:2403.18403.

Method: LLM-based identification of cryptographic functions in stripped (symbol-free) binaries, a harder problem than source-code analysis. Relevant for software supply chain security: detecting which dependencies use vulnerable crypto without source access.

Significance: Binary-level crypto detection is essential for third-party library analysis and embedded systems. If you cannot audit source, binary analysis is the only option. Directly relevant to IoT and legacy system PQC migration.

Limitations: Accuracy degrades significantly with obfuscated binaries. LLM hallucinations are more dangerous in binary analysis (misidentifying a function as implementing AES when it does not has serious security consequences).

Cluster B: Fine-Tuned LLMs for Secure Code Generation

This cluster addresses a different problem: not detecting existing vulnerable code, but generating new code that is secure from the start. Several research directions exist: white-box fine-tuning on secure/vulnerable datasets, preference learning (RLHF-style), and constrained decoding.

B1. SVEN & SafeCoder (He et al., 2023/2024)

He, J., & Vechev, M. (2023). Large Language Models for Code: Security Hardening and Adversarial Testing (SVEN). CCS 2023 (Distinguished Paper Award). SafeCoder: He, J., Vero, M., Krasnopolka, G., & Vechev, M. (2024). Instruction Tuning for Secure Code Generation. ICML 2024.

Method: SVEN employs prefix-based fine-tuning; it prepends "security-steering" prefixes learned from datasets of secure vs. insecure code, without modifying model weights. SafeCoder extends this to instruction-tuning on a curated dataset. Both are white-box methods requiring model weight access.

Significance: SVEN achieved CCS 2023 Distinguished Paper status, the highest quality marker in the field. The insight that security can be steered through learned prefixes (rather than full fine-tuning) is architecturally important for resource-constrained research groups.

Limitations: Evaluated on general CWEs, not cryptographic patterns specifically. White-box requirement limits deployment to open-weight models (Llama, Mistral) and excludes GPT-4 and Claude APIs. A later evaluation (Dai et al., 2025) showed that secure code generation techniques including SVEN often degrade functional correctness, in some cases by more than 50%.

B2. Fine-Tuning LLMs for Secure Code Generation (Li et al., 2024)

Li, J., et al. (2024). Fine Tuning Large Language Model for Secure Code Generation. IEEE/ACM FORGE 2024.

Method: Supervised fine-tuning (SFT) of CodeLlama on a dataset of paired (vulnerable, fixed) code examples. Evaluated on SecurityEval benchmark.

Significance: Direct evidence that domain-specific fine-tuning on security data reduces vulnerability rate. Provides a baseline methodology for fine-tuning on cryptographic code specifically.

Limitations: Generic security dataset, not cryptography-specific. No evaluation on PQC API usage. The CWE categories evaluated (buffer overflow, injection) are different from cryptographic misuse patterns.

B3. CyberLLMInstruct: Fine-Tuning Safety Trade-offs (ElZemity et al., 2025)

ElZemity, A., Arief, B., & Li, S. (2025). CyberLLMInstruct: A New Dataset for Analysing Safety of Fine-Tuned LLMs Using Cyber Security Data. AISEC '25 (ACM), 77–88.

Method: Curates a large dataset for instruction-tuning LLMs on cybersecurity tasks, and empirically demonstrates the trade-off between performance gains (on CyberMetric benchmark) and safety degradation through fine-tuning.

Significance: Critical finding for this research: fine-tuning on security data can make LLMs more capable cybersecurity assistants but also more willing to generate harmful outputs. This is directly relevant if we attempt to fine-tune a model on PQC migration code: we need adversarial testing to verify the fine-tuned model does not generate dangerous cryptographic implementations.

Limitations: Dataset is general cybersecurity, not PQC-specific. The safety evaluation framework (OWASP Top 10) does not include quantum-specific threat categories.

B4. Rethinking Secure Code Generation Evaluation (2025)

Dai, S.-C., Xu, J., & Tao, G. (2025). Rethinking the Evaluation of Secure Code Generation. arXiv:2503.15554.

Method: Meta-study evaluating four leading secure code generation techniques (SVEN, SafeCoder, CodeGuard+, PromSec). Key finding: techniques improve security metrics but often sacrifice functional correctness; they may simply remove security-critical code or produce incomplete outputs rather than genuinely secure alternatives.

Significance: A sobering evaluation. For PQC migration, generating incorrect but "secure" code is potentially worse than the original vulnerable code: a broken ML-KEM implementation is both insecure and non-functional. Any LLM-based PQC migration tool must be evaluated on both security and functional correctness simultaneously.

B5. ZeroLeak: Side-Channel Patching with LLMs (Tol & Sunar, 2023)

Tol, M. C., & Sunar, B. (2023). ZeroLeak: Using LLMs for Scalable and Cost Effective Side-Channel Patching. arXiv:2308.13062.

Method: Uses LLMs in a zero-shot setting to generate constant-time implementations of cryptographic algorithms, patching timing side-channels. Generates algorithm implementations function-by-function to preserve coherence and enable modular testing.

Significance: Side-channel resistance is a critical requirement for PQC implementations (ML-KEM and ML-DSA are vulnerable to timing attacks if naively implemented). ZeroLeak demonstrates that LLMs can generate constant-time code with appropriate prompting, which is directly applicable to PQC implementation security.

Limitations: Zero-shot only. Formal verification of constant-time properties is not performed; the output is not formally proven side-channel-free.

Cluster C: LLM Benchmarking in Cryptographic Domains

Before LLMs can be trusted for cryptographic tasks, we need to know what they actually understand. This cluster provides empirical characterization of LLM cryptographic capabilities and limitations.

C1. CipherBank (Li et al., 2025)

Li, Y., et al. (2025). CipherBank: Exploring the Boundary of LLM Reasoning Capabilities through Cryptography Challenges. arXiv:2504.19093.

Method: Benchmark of 2,358 problems covering cryptographic decryption tasks across 3 categories and 9 algorithms. Evaluates 18 LLMs including GPT-4o, DeepSeek-V3, o1, DeepSeek-R1. Focus on reasoning ability under cryptographic constraints.

Key Findings: Reasoning-focused models (o1, DeepSeek-R1) outperform chat models across all cipher types. However, all models struggle with custom/novel cryptographic techniques; models rely on pattern matching from training data rather than genuine cryptographic reasoning. Gap between memorized knowledge and dynamic reasoning is large.

Significance for PQC Research: LLMs' inability to reason about novel cryptographic constructs suggests they will struggle with post-quantum schemes they have seen less training data on (ML-KEM, SLH-DSA). A fine-tuned model with PQC-specific training data may be required.

C2. AICrypto (Wang et al., 2025)

Wang, Y., et al. (2025). AICrypto: A Comprehensive Benchmark for Evaluating Cryptography Capabilities of Large Language Models. arXiv:2507.09580.

Method: Comprehensive benchmark with 135 multiple-choice questions, 150 CTF challenges, and 30 formal proof problems. Covers knowledge memorization, vulnerability exploitation, and formal reasoning. Human expert assessment correlation: high (automated graders validated).

Key Findings: LLMs match or exceed human experts on MCQs (knowledge memorization) but fall significantly short on CTF challenges and proof problems. Mathematical comprehension, as opposed to

memorization, is the limiting factor. Models lack dynamic numerical reasoning ability for complex cryptographic analysis.

Significance: Establishes the current ceiling of LLM cryptographic capability. For PQC migration tooling, the LLM's role should be restricted to pattern recognition and classification tasks (where it excels), not formal security proofs (where it fails). This directly shapes the architecture of any proposed tool.

C3. Benchmarking LLMs for Cryptanalysis (Maskey et al., 2025)

Maskey, U., et al. (2025). Benchmarking Large Language Models for Cryptanalysis and Mismatched-Generalization. arXiv:2505.24621.

Method: Evaluates LLMs on classical cryptanalysis tasks, with a focus on generalization failures: cases where models appear to solve a cryptographic problem but are actually exploiting spurious correlations rather than cryptographic structure.

Significance: "Mismatched generalization" (LLMs solving the test but failing on structurally identical problems with different surface features) is a fundamental concern for any security-critical LLM application. A PQC migration tool that appears to correctly classify cryptographic usage but generalizes poorly to real enterprise codebases is dangerous.

Cluster D: Multi-Agent LLM Systems for Security

Single-LLM approaches have fundamental limitations: context window constraints, hallucination, and single-point-of-failure. Multi-agent architectures distribute tasks across specialized agents, enabling more robust and scalable analysis.

D1. Quantigence: Multi-Agent Framework for Quantum Security (2025)

Alquwayfili, A. (2025). Quantigence: A Multi-Agent AI Framework for Quantum Security Research. arXiv:2512.12989.

Method: Multi-agent LLM framework specifically designed for quantum security research tasks. Decomposes research objectives into specialized agent roles (Cryptographic Analyst, Threat Modeler, Standards Specialist, and Risk Assessor) coordinated by a supervisory agent. Integrates external knowledge sources (NIST PQC database, NVD, arXiv/IACR) via the Model Context Protocol (MCP) and prioritizes vulnerabilities using a Quantum-Adjusted Risk Score that extends Mosca's Theorem.

Significance: The only multi-agent framework explicitly targeting quantum security. The external knowledge integration with PQC sources is directly relevant; a similar architecture could be applied to PQC migration tooling, where agents specialize in crypto detection, risk scoring, and migration suggestion.

Limitations: Preliminary work with no rigorous evaluation on standardized benchmarks. The quantum security "research" framing is broad; specific evaluation on PQC migration tasks is absent.

D2. Collaborative LLMs (Debate or Vote) for Crypto Detection (2025)

Choi, H.K., et al. (2025). Debate or Vote: Which Yields Better Decisions in Multi-Agent LLMs? Referenced in arXiv:2603.07204.

Method: Disentangles multi-agent debate into its two components, majority voting and inter-agent debate, and assesses their respective contributions across seven NLP benchmarks. Applied to cryptographic package detection by Hirsch et al. (2026).

Significance: Finds that majority voting alone accounts for most of the performance gains attributed to multi-agent debate, and that debate alone does not improve expected correctness. This informs the architecture of a PQC migration tool: ensemble voting is the more reliable and efficient choice for the detection and classification steps.

D3. LLM-Based Multi-Agent Systems for Software Engineering (He et al., 2025)

He, J., Treude, C., & Lo, D. (2025). LLM-based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. ACM TOSEM 2025.

Method: Comprehensive survey of multi-agent LLM approaches in SE. Identifies design patterns including RAG, cross-reflection, human-reflection, tool-agent registry, and debate-based cooperation.

Significance: Provides the theoretical design vocabulary for building a multi-agent PQC migration system. The "tool-agent registry" pattern (agents using external tools like static analyzers) is directly applicable: a PQC migration agent could call CryptoGuard, SAST tools, and NIST API endpoints as external tools.

Cluster E: LLM-Driven PQC Hardware Acceleration

A separate but important thread: using LLMs not to find or fix vulnerabilities, but to accelerate implementation of PQC algorithms in hardware.

E1. Accelerating PQC via LLM-Driven Hardware-Software Co-Design (2026)

Liao, Y., Adegbiya, T., & Lysecky, R. (2026). Accelerating Post-Quantum Cryptography via LLM-Driven Hardware-Software Co-Design. ISQED 2026. arXiv:2602.09410.

Method: LLM framework for hardware-software co-design of PQC on FPGAs, focused on FALCON (FN-DSA). Process: (1) profile C/C++ PQC implementation, (2) LLM-driven hardware/software partitioning, (3) LLM generates Verilog RTL for FPGA. Reports up to 2.6x kernel speedup over conventional High-Level Synthesis (HLS) approaches with human-in-the-loop refinement, with trade-offs in resource utilization and power consumption.

Significance: First paper to use LLMs for PQC FPGA acceleration. As PQC algorithms are computationally heavier than RSA/ECC, hardware acceleration is essential for IoT and embedded deployments. If LLMs can automate FPGA design for PQC, the deployment barrier for constrained devices is significantly lowered.

Limitations: Evaluated only on FALCON, the NIST PQC pipeline algorithm with the most complex implementation. Generalization to ML-KEM and ML-DSA not demonstrated. LLM-generated Verilog has known correctness issues; formal verification is not performed.

E2. TFHE-Coder: LLM for Homomorphic Encryption (2025)

Kumar, M., Xue, J., Zheng, M., & Lou, Q. (2025). TFHE-Coder: Evaluating LLM-Agentic Fully Homomorphic Encryption Code Generation. arXiv:2503.12217.

Method: Agentic LLM system for generating Fully Homomorphic Encryption (FHE) code using the TFHE library. Evaluates correctness and efficiency of LLM-generated FHE implementations.

Significance: FHE is a post-quantum-compatible technique with enormous complexity barriers. If LLMs can correctly generate FHE code, they may be able to assist with ML-KEM/ML-DSA implementations. The agentic architecture (self-correcting through iterative testing) is the right model for complex cryptographic code generation.

Cluster F: Quantum-Safe Software Engineering Frameworks (Zhang et al. & EST Lab)

This cluster contains the most directly applicable work to the summer research project. Dr. Zhang's group at UMBC has pioneered the "quantum-safe software engineering" (QSSE) research agenda, which treats PQC migration as a software engineering discipline rather than a purely cryptographic one.

F1. Making Existing Software Quantum Safe: IBM Db2 Case Study (Zhang et al., 2023)

Zhang, L., Miransky, A., Rjaibi, W., Stager, G., Gray, M., & Peck, J. (2023). Making Existing Software Quantum Safe: A Case Study on IBM Db2. *Information and Software Technology*, 161, 107249. arXiv:2110.08661.

Method: Industrial case study on migrating IBM Db2 (a large production DBMS) to PQC. Systematically identifies: where classical cryptography lives in a production codebase, the testing challenges introduced by migration, and architectural lessons for crypto-agility.

Key Findings: Cryptographic dependencies are distributed across codebase layers, not localized. Existing test suites are insufficient for migration correctness verification. Crypto-agility (the ability to swap algorithms without full rewrites) must be designed in from the beginning.

Significance: Provides the empirical template for any codebase-level PQC migration study. The finding that test suites are insufficient is directly actionable: developing better test generation strategies for PQC migration is an open research contribution.

F2. Survey of PQC Support in Cryptographic Libraries (Ahmed, Zhang & Gangopadhyay, 2025)

Ahmed, N., Zhang, L., & Gangopadhyay, A. (2025). A Survey of Post-Quantum Cryptography Support in Cryptographic Libraries. *IEEE QCE* 2025.

Method: Empirical survey of 9 open-source cryptographic libraries (OpenSSL, wolfSSL, BoringSSL, LibreSSL, Bouncy Castle, libsodium, etc.) for NIST PQC algorithm support, as of early 2025.

Key Findings: PQC library support is highly uneven. OpenSSL, the most widely deployed library, has incomplete PQC support. Bouncy Castle (Java/C#) has the most comprehensive coverage. Several libraries have no ML-KEM support. Most software depending on these libraries is effectively quantum-vulnerable regardless of developer intent.

Significance: Establishes the "where" of the migration problem. Any LLM-based migration tool must know which library versions support which PQC algorithms; this survey is the knowledge base for that.

F3. Toward Quantum-Safe Software Engineering: AQuA Framework (Zhang, 2026)

Zhang, L. (2026). Toward Quantum-Safe Software Engineering: A Vision for Post-Quantum Cryptography Migration. ICSE 2026 Poster. arXiv:2602.05759.

Method: Vision paper articulating QSSE as a research discipline and introducing the AQuA (Automated Quantum-safe Analysis) framework: a four-layer architecture for PQC migration tooling covering detection, classification, refactoring, and verification.

Key Contributions: (1) Formalizes QSSE as distinct from cryptographic research; (2) AQuA framework provides a blueprint for migration tools; (3) Identifies crypto-agility as an architectural property requiring proactive design; (4) Calls for collaboration between SE, security, and cryptography communities.

Significance: This is the directly supervising framework for the summer research. The AQuA framework's detection and classification layers are where LLMs are most applicable, and most underspecified. The research contribution opportunity is to instantiate the detection/classification layers with LLM-based approaches and evaluate their performance.

F4. Quantum Advantage and Y2K Bug: A Comparison (Zhang, Miranskyy & Rjaibi, 2021)

Zhang, L., Miranskyy, A., & Rjaibi, W. (2021). Quantum Advantage and the Y2K Bug: A Comparison. IEEE Software, 38(2), 80–87.

Framing paper drawing a structural analogy between the Y2K software remediation crisis and the quantum cryptography migration challenge. Argues that quantum migration is structurally similar to Y2K (distributed, systemic, hard-deadline) but technically harder and with a more uncertain timeline. Establishes the urgency framing for QSSE research.

F5. Is Your Quantum Program Bug-Free? (Miranskyy, Zhang & Doliskani, 2020)

Miranskyy, A., Zhang, L., & Doliskani, J. (2020). Is Your Quantum Program Bug-Free? ICSE-NIER 2020, 29–32 (Distinguished Paper Award).

Foundational quantum software engineering paper examining correctness and testing challenges for quantum programs. Establishes the testing methodology thread that connects to the migration verification gap identified in the Db2 case study.

4. Research Synthesis: What Do We Know?

Synthesizing across the six clusters, the following picture emerges:

4.1 What LLMs Can Do Well (in the cryptographic domain)

- Pattern-based crypto detection: LLMs reliably identify known cryptographic API calls, library imports, and usage patterns in source code (Firouzi & Ghafari 2024; Shaw 2026; FoC 2024). This is essentially a sophisticated text classification task where LLMs excel.
- Contextual enrichment: LLMs add semantic context that pure regex/AST tools lack: understanding why a cryptographic primitive is used, not just that it is used (IRIS 2025; Shaw 2026). This is critical for risk prioritization.

- Knowledge retrieval: RAG-augmented LLMs can accurately answer questions about NIST standards, library support, and migration guidance (Quantigence 2025; Collaborative LLMs 2026).
- Constant-time code generation: With appropriate prompting, LLMs can generate side-channel-resistant cryptographic implementations (ZeroLeak 2023).
- MCQ-level cryptographic knowledge: LLMs match human experts on multiple-choice cryptographic knowledge questions (AICrypto 2025; CipherBank 2025).

4.2 What LLMs Cannot Do Well (yet)

- Dynamic cryptographic reasoning: LLMs fail on novel cryptographic problems requiring computation rather than pattern matching. They cannot verify security proofs or analyze new algorithm constructions reliably (AICrypto 2025; CipherBank 2025; Maskey 2025).
- Migration correctness verification: No existing system can automatically verify that a PQC-migrated codebase is functionally equivalent to its predecessor; this is the gap identified by Zhang et al. (2023).
- Large-codebase analysis: All evaluated systems are tested on small libraries (thousands of LOC). Enterprise codebases (millions of LOC, like Db2) remain unevaluated. Context window constraints are a hard limit.
- Generalization without fine-tuning: Zero-shot LLMs applied to PQC-specific tasks will struggle because training data for ML-KEM, ML-DSA, and SLH-DSA is orders of magnitude sparser than for RSA/AES.
- Functional correctness under security constraints: Secure fine-tuning often degrades functional correctness, a dangerous trade-off in cryptographic contexts (Rethinking 2025).

4.3 The Critical Gap

No existing work applies LLMs specifically to the PQC migration problem as defined by Zhang et al.'s AQuA framework: automated detection of quantum-vulnerable code, classification by risk, and suggestion of PQC-compliant replacements, evaluated on real production codebases with both precision/recall metrics AND functional correctness verification.

Shaw (2026) comes closest but lacks: fine-tuning on PQC-specific data, evaluation on enterprise-scale codebases, functional correctness verification, and integration with the broader QSSE methodology. The collaborative LLMs paper (2026) addresses detection but not classification, risk scoring, or migration suggestion. Zhang's AQuA framework provides the theoretical structure but no LLM instantiation.

5. Identified Problems in the Literature

Problem	Evidence	Impact
No PQC-specific fine-tuned LLM	CipherBank (2025) shows LLMs fail on	Detection/migration accuracy is

exists	novel crypto; ZeroLeak uses zero-shot only	limited by training data sparsity for PQC schemes
No migration correctness verification	Zhang et al. (2023): existing test suites insufficient	A migrated codebase may be functionally broken even if classically-vulnerable primitives are removed
Small-scale evaluation only	All papers test on <10K LOC libraries; no enterprise evaluation	Tools may fail at scale; no evidence of enterprise applicability
Security-correctness trade-off unresolved	Rethinking (2025): secure fine-tuning degrades functional correctness	Cannot deploy a tool that produces secure but broken migrations
Library ecosystem gap	Ahmed et al. (2025): OpenSSL PQC support is incomplete	LLM tools must account for library-level constraints, not just algorithm-level
Multi-agent PQC systems are preliminary	Quantigence (2025) lacks rigorous evaluation	No validated multi-agent architecture for PQC migration exists

6. What I Understand and What I Propose to Do

6.1 My Understanding of the Field

The literature makes clear that the PQC migration problem is fundamentally a software engineering challenge, not a cryptography challenge. The cryptography is solved; NIST has finalized standards. What is unsolved is: how do organizations systematically find, assess, and replace quantum-vulnerable code across millions of lines of production software, in the absence of dedicated cryptographic expertise on most development teams?

LLMs represent the most promising automation technology for this problem, but existing applications are either: (a) too narrow (focused on detection only, ignoring migration and verification), (b) unevaluated at enterprise scale, (c) lacking PQC-specific training, or (d) disconnected from the systematic SE methodology that Zhang et al.'s AQuA framework provides.

The intersection of LLM capabilities (contextual code understanding, RAG-augmented knowledge retrieval, multi-agent orchestration) and the AQuA framework's structured migration pipeline is exactly where a research contribution is possible.

6.2 Proposed Research: LLM-Augmented PQC Migration Analysis

Working Title: "LLM-Augmented Detection and Classification of Quantum-Vulnerable Code: Toward the AQuA Framework's Detection Layer"

Research Objective

Design, implement, and evaluate an LLM-augmented tool that operationalizes the detection and classification layers of Zhang's AQuA framework, specifically: (1) identifying quantum-vulnerable cryptographic usages in Python and Java codebases, (2) classifying each usage by risk (high/medium/low, informed by NIST IR 8547 and CNSA 2.0 timelines), and (3) suggesting PQC-compliant replacements using the library support data from Ahmed et al. (2025).

Methodology

1. **Crypto-Vulnerable Pattern Extraction:** Build a comprehensive taxonomy of quantum-vulnerable API patterns across Python (cryptography, PyCryptodome, python-rsa) and Java (JCA, Bouncy Castle classical APIs), extending Shaw's 15-class taxonomy with PQC-specific context (e.g., which library version to migrate to, what FIPS standard replaces each primitive).
2. **LLM Pipeline Design:** Implement a hybrid detection pipeline: regex/AST for candidate identification (precision), LLM contextual enrichment for classification (recall + context), RAG-augmented suggestion for migration recommendations (using Ahmed et al.'s library support data as the knowledge base).
3. **Evaluation Dataset:** Evaluate on (a) the open-source libraries from Shaw (2026) for comparison, and (b) a new evaluation set of larger open-source projects with known cryptographic dependencies.
4. **Dual Metric Evaluation:** Measure both security metrics (precision/recall on vulnerability detection) and functional metrics (do the suggested replacements compile and pass existing tests?), directly addressing the gap identified in Rethinking (2025).
5. **Comparison to Baselines:** Compare to: (a) pure regex/SAST (CryptoGuard baseline), (b) zero-shot GPT-4, and (c) Shaw's Quantum-Safe Code Auditor.

Connection to Zhang Lab

- Extends the AQuA framework with a concrete LLM instantiation: the first empirical evaluation of LLMs in the detection/classification layers.
- Directly builds on the IBM Db2 case study methodology (systematic crypto identification) and the library survey (replacement mapping).
- Addresses the "insufficient test suite" gap from Zhang et al. (2023) by evaluating functional correctness of suggested replacements.
- Positions for a research paper contribution to SANER 2027 or QCE 2027.

Research Phase Timeline (10 Weeks, July–September 2026)

Weeks	Milestone
1–2	Deep literature review + taxonomy of quantum-vulnerable API patterns (Python + Java)
3–4	Implement regex/AST detection layer + curate evaluation dataset
5–6	Integrate LLM contextual enrichment + RAG knowledge base (NIST standards, Ahmed et al. library data)
7	Run evaluation: precision/recall + functional correctness. Compare to baselines.
8–9	Analysis, write-up, research paper draft

7. Next Steps

Immediately prior to the research phase kickoff in July, the following actions will position this research for a productive start:

1. Verify with Dr. Zhang: Is the detection/classification layer of AQuA the intended summer focus, or is there a specific codebase/system already identified for the case study?
2. Complete the pre-kickoff reading plan (see Section 8), particularly the IBM Db2 paper and the ICSE 2026 vision paper.
3. Set up the development environment: Python, Java, static analysis tools (CodeQL, CryptoGuard), and an open-weight LLM (Llama 3.1 or Mistral 7B for local inference).
4. Pull the open-source evaluation libraries: python-rsa, python-ecdsa, python-jose from Shaw (2026) as the initial evaluation dataset.
5. Draft the quantum-vulnerable API taxonomy in consultation with Dr. Zhang before coding begins.

8. Pre-Kickoff Reading Plan

Week 1 – PQC Foundations + Zhang Lab

- Bernstein & Lange (2017), Nature – best single PQC survey
- NIST FIPS 203/204/205 (2024) – executive summaries
- Zhang et al. (2023) IBM Db2 – arXiv:2110.08661 [PRIORITY]
- Zhang (2026) ICSE AQuA framework – arXiv:2602.05759 [PRIORITY]

Week 2 – LLM × Crypto Literature

- Shaw (2026) Quantum-Safe Code Auditor – arXiv:2604.00560
- Ahmed, Zhang, Gangopadhyay (2025) PQC Library Survey – QCE 2025
- AICrypto (2025) – arXiv:2507.09580 (LLM capabilities baseline)
- CipherBank (2025) – arXiv:2504.19093 (LLM reasoning gaps)
- Firouzi & Ghafari (2024) ESEM – ChatGPT vs. SAST for crypto misuse
- He et al. (2025) TOSEM – Multi-agent LLM survey for SE

9. Full References

- Ahmed, N., Zhang, L., & Gangopadhyay, A. (2025). A Survey of Post-Quantum Cryptography Support in Cryptographic Libraries. IEEE QCE 2025.
- Alquwayfili, A. (2025). Quantigence: A Multi-Agent AI Framework for Quantum Security Research. arXiv:2512.12989.

- Bernstein, D. J., & Lange, T. (2017). Post-quantum cryptography. *Nature*, 549(7671), 188–194.
- Choi, H. K., Zhu, X., & Li, Y. (2025). Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models? *arXiv:2508.17536*.
- Dai, S.-C., Xu, J., & Tao, G. (2025). Rethinking the Evaluation of Secure Code Generation. *arXiv:2503.15554*.
- ElZemity, A., Arief, B., & Li, S. (2025). CyberLLMInstruct: A New Dataset for Analysing Safety of Fine-Tuned LLMs Using Cyber Security Data. *AISeC '25 (ACM)*, 77–88.
- Firouzi, E., & Ghafari, M. (2024). ChatGPT's Potential in Cryptography Misuse Detection: A Comparative Analysis with Static Analysis Tools. *ESEM 2024*.
- Firouzi, E., & Ghafari, M. (2024). Time to Separate from StackOverflow and Match with ChatGPT for Encryption. *Journal of Systems and Software*, 216.
- Firouzi, E., & Ghafari, M. (2026). Can Generative AI Detect and Fix Real-World Cryptographic Misuses? *Journal of Systems and Software*, 232.
- He, J., & Vechev, M. (2023). Large Language Models for Code: Security Hardening and Adversarial Testing (SVEN). *CCS 2023 (Distinguished Paper Award)*.
- He, J., Treude, C., & Lo, D. (2025). LLM-Based Multi-Agent Systems for Software Engineering. *ACM TOSEM 2025*.
- Hirsch, E., Raab, K., Bauer, T. J., & Loebenberger, D. (2026). Detecting Cryptographically Relevant Software Packages with Collaborative LLMs. *ICISSP 2026. arXiv:2603.07204*.
- Kumar, M., Xue, J., Zheng, M., & Lou, Q. (2025). TFHE-Coder: Evaluating LLM-Agentive Fully Homomorphic Encryption Code Generation. *arXiv:2503.12217*.
- Li, J., et al. (2024). Fine Tuning Large Language Model for Secure Code Generation. *IEEE/ACM FORGE 2024*.
- Li, Y., et al. (2025). CipherBank: Exploring the Boundary of LLM Reasoning Capabilities through Cryptography Challenges. *arXiv:2504.19093*.
- Li, Z., Dutta, S., & Naik, M. (2025). IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. *ICLR 2025*.
- Liao, Y., Adegbiya, T., & Lysecky, R. (2026). Accelerating Post-Quantum Cryptography via LLM-Driven Hardware-Software Co-Design. *ISQED 2026. arXiv:2602.09410*.
- Maskey, U., et al. (2025). Benchmarking Large Language Models for Cryptanalysis and Mismatched-Generalization. *arXiv:2505.24621*.
- Miranskyy, A., Zhang, L., & Doliskani, J. (2020). Is Your Quantum Program Bug-Free? *ICSE-NIER 2020*, 29–32 (Distinguished Paper Award).
- NIST (2024). FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.
- NIST (2024). FIPS 204: Module-Lattice-Based Digital Signature Algorithm Standard.
- NIST (2024). FIPS 205: Stateless Hash-Based Digital Signature Algorithm Standard.
- NIST (2024). NIST IR 8547 (ipd): Transition to Post-Quantum Cryptography Standards.
- Shang, X., Chen, G., Cheng, S., Zhang, Y., Zhang, W., & Yu, N. (2024). FoC: Figure out the Cryptographic Functions in Stripped Binaries with LLMs. *arXiv:2403.18403*.
- Shaw, A. (2026). Quantum-Safe Code Auditing: LLM-Assisted Static Analysis and Quantum-Aware Risk Scoring for Post-Quantum Cryptography Migration. *arXiv:2604.00560*.
- Shor, P. W. (1997). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5), 1484–1509.
- Tol, M. C., & Sunar, B. (2023). ZeroLeak: Using LLMs for Scalable and Cost Effective Side-Channel Patching. *arXiv:2308.13062*.
- Wang, Y., et al. (2025). AICrypto: A Comprehensive Benchmark for Evaluating Cryptography Capabilities of Large Language Models. *arXiv:2507.09580*.
- Zhang, L. (2026). Toward Quantum-Safe Software Engineering: A Vision for Post-Quantum Cryptography Migration. *ICSE 2026 Poster. arXiv:2602.05759*.
- Zhang, L., Miranskyy, A., & Rjaibi, W. (2021). Quantum Advantage and the Y2K Bug: A Comparison. *IEEE Software*, 38(2), 80–87.
- Zhang, L., Miranskyy, A., Rjaibi, W., Stager, G., Gray, M., & Peck, J. (2023). Making Existing Software Quantum Safe: A Case Study on IBM Db2. *Information and Software Technology*, 161, 107249. *arXiv:2110.08661*.

